

Project 1 — Building a Search Engine from Scratch

Course: Multimedia Information Retrieval & Computer Vision, University of Pisa

Academic year: 2026/2027

Instructor: Nicola Tonellotto (nicola.tonellotto@unipi.it)

Version: 0.3

1. Goal

You will write, in a Google Colab notebook, a program that builds an inverted index from a collection of text documents, and a program that processes ranked queries over that index. The engine must index the full MS MARCO passage collection (8.8 million passages) within a free Colab runtime and be evaluated on standard test queries.

In short, you must:

1. read and process the 8.8 million passages of the collection;
2. build an inverted index on disk, using a bounded amount of memory;
3. answer ranked conjunctive (AND) and disjunctive (OR) queries with TF-IDF and BM25;
4. evaluate effectiveness and efficiency on three standard query sets;
5. submit a single Colab notebook containing code, tests, results, and report.

Decompose the system into components. For example, indexing can be carried out by one component that parses documents and writes intermediate postings to disk in unsorted or partially sorted form, one that sorts and merges the postings, and one that turns the merged postings into the final index and lookup structures. Build a test collection of a few documents early on, and write regression tests that check every component on it.

Using an existing search library for any of the components below is not allowed (see [Section 3](#)).

2. Learning objectives

By the end of the project you should be able to:

Objective	Lecture notes
Turn raw text into index terms and justify each processing step	Ch. 2
Build an inverted index for a collection that does not fit in memory	Ch. 3
Process conjunctive and disjunctive ranked queries over posting lists	Ch. 5
Implement and compare TF-IDF and BM25 ranking	Ch. 6, Ch. 8
Evaluate effectiveness and efficiency with sound methodology	Ch. 9
(Extensions) Compress posting lists and quantify the space-time trade-off	Ch. 4
(Extensions) Compare query-processing strategies, and speed them up with skipping and dynamic pruning	Ch. 5

Objective	Lecture notes
(Extensions) Improve ranking with feedback and query expansion	Ch. 10

3. Rules

- **Teams.** From three to four students, with no exceptions. A student belongs to one team only. Every member must be able to explain every part of the code and of the report.
- **Platform.** The project must be developed and run in **Google Colab**. Everything you report must be reproducible on a **free CPU runtime**; no GPU is needed. If you use a paid plan while developing, state it, and make sure the notebook still runs on a free runtime.
- **Language.** Python, in the version provided by Colab. Performance-critical loops may be compiled with Cython (`%cython`), Numba, or small C extensions, provided they build and run inside Colab and you wrote them yourself. `numpy` and the whole standard library (`re`, `unicodedata`, `array`, `struct`, `mmap`, `heapq`, `tarfile`, `pickle`, ...) are available.
- **Allowed libraries.**
 - `ir_datasets` for the test queries and relevance judgements only; the collection must be downloaded and read by your own code ([Section 4](#));
 - a stemmer implementing the Porter algorithm or its Snowball English successor, e.g., `PyStemmer` or `nltk.stem`;
 - any English stop-word list available on the Web; cite its source;
 - `ir_measures` for effectiveness measures, `scipy.stats` for significance tests;
 - `numpy`, `pandas` (for tables of results, not for the index), `tqdm`, `psutil`, plotting libraries.

Tokenisation and normalisation must be your own code; the standard library (`re`, `unicodedata`, `str` methods) is enough. Any library not listed here may be used only for tasks unrelated to indexing, compression, query processing, and scoring.
- **Forbidden libraries and storage.**
 - Any library that provides indexing, compression, or ranking, including `Lucene`, `Pyserini`, `PyTerrier`, `Whoosh`, `Tantivy`, `rank_bm25`, `bm25s`, `Elasticsearch/OpenSearch`, and `scikit-learn` (`TfidfVectorizer` and similar).
 - Databases and key-value stores for postings, including `SQLite`, `LMDB`, `shelve`, and `dbm`; `scipy.sparse` matrices as the index.
 - General-purpose compressors (`gzip`, `zlib`, `bz2`, `lzma`, `zstd`, ...) applied to the index. Using `tarfile` or `gzip` to read the compressed collection is fine.

When in doubt, ask before using a library.
- **AI assistants.** Allowed, but every use must be declared in the report, and you are fully accountable for, and must be able to explain, every line you submit.
- **Plagiarism.** Code shared between teams, or copied without attribution from public repositories or from projects of previous years, leads to the project being rejected.

4. Data

4.1 Collection

The MS MARCO passage collection, available from the [TREC Deep Learning 2020 page](#): in the section *Passage ranking dataset*, the first link of the table is `collection.tar.gz` (about 1 GB compressed). Download it from within the notebook, e.g., with `wget`. The archive contains a single file, `collection.tsv` (about 3 GB), with 8,841,823 passages, one per line, in the format

```
<pid>\t<text>\n
```

where `<pid>` is the passage identifier (an integer) and `<text>` the passage content.

Messy input. The text may include non-English languages and non-ASCII characters, and may contain errors such as empty passages, malformed lines, and malformed characters. Handle them as follows:

- decode the input as UTF-8, replacing invalid byte sequences (`errors="replace"`) rather than failing;
- skip lines without a tab or whose pid is not an integer, and count them;
- keep passages whose text is empty after processing: they receive a docid and have length 0;
- you are not required to repair mis-encoded text, e.g., `â€™` instead of an apostrophe.

Report the number of skipped lines and of empty passages.

Identifiers. A *pid* is the identifier in the collection file. A *docid* is the integer your index assigns to each passage, consecutively from 0 in the order in which passages are read. In MS MARCO docids and pids may coincide, but your document table must store the mapping explicitly, and run files must always contain pids.

4.2 Test queries and relevance judgements

Load them with `ir_datasets`, e.g., `ir_datasets.load("msmarco-passage/trec-dl-2019/judged")`, then `queries_iter()` and `qrels_iter()`.

Query set	<code>ir_datasets</code> identifier	Queries	Judgements
TREC DL 2019	<code>msmarco-passage/trec-dl-2019/judged</code>	43	graded, 0–3
TREC DL 2020	<code>msmarco-passage/trec-dl-2020/judged</code>	54	graded, 0–3
MS MARCO dev (small)	<code>msmarco-passage/dev/small</code>	6,980	binary, about one relevant passage per query

On TREC DL, relevance levels are 0 (irrelevant), 1 (related), 2 (highly relevant), and 3 (perfectly relevant). Use the `judged` identifiers: they contain only the queries that have judgements. Queries must go through the same text processing as passages.

4.3 Colab in practice

- A free runtime has about 12 GB of RAM and two CPU cores, and is disconnected after a period of inactivity or after a maximum session length. Anything stored only on the runtime's local disk (`/content`) is lost when the session ends.
- Store what must survive a session, the final index above all, on your Google Drive. A free Google account has 15 GB of Drive space: do not keep the uncompressed collection or the intermediate runs of indexing there longer than needed.
- Reading from Google Drive is much slower than reading from local disk. At the start of a session, copy the index to `/content` and run queries from there.
- Use the first 100,000 lines of `collection.tsv` as a **development subset** while debugging. Its evaluation scores are meaningless, because most relevant passages are outside the subset.

5. Requirements

5.1 Core (mandatory for every team)

A correct and complete core is the threshold for a pass.

C1 – Document processing. Implement a text-processing function applied identically to passages and queries, with the following steps:

1. case normalisation;
2. punctuation removal;
3. tokenisation;
4. stop-word removal;
5. stemming.

A reasonable default for the first three steps is: lower-case the text, replace every character that is not a letter or a digit (not `c.isalnum()`) with a space, and split on whitespace. You may choose differently, e.g., adding Unicode normalisation, but justify your choices in the report. Stop-word removal and stemming must each be switchable by a flag in the configuration cell (Section 6), so that you can measure their effect.

C2 — Inverted index. Build the inverted index with a memory-bounded algorithm, either single-pass in-memory indexing or blocked sort-based indexing:

- buffer postings in memory and write them to disk as a sorted intermediate run whenever the buffer exceeds a configurable size;
- merge all intermediate runs into a single final index;
- use this algorithm even if your implementation would fit in memory: it must work unchanged, only more slowly, with a buffer ten times smaller.

The final index consists of:

- a **lexicon** storing, for each term, its document frequency, its collection frequency (total number of occurrences), and the position (offset and length) of its posting list in the postings file;
- the **posting lists**, each holding the docids of the documents containing the term, in increasing order, with the corresponding term frequencies;
- a **document table** mapping each docid to its pid and to its length;
- the **collection statistics**: number of documents N and average document length $|d|_{\text{avg}}$.

The **length** $|d|$ of a document is its number of index terms, i.e., of tokens after processing, stop words excluded when stop-word removal is enabled. Documents of length 0 count in N and in the average.

Storage rules:

- all structures must be stored on disk at the end of indexing;
- store all posting lists in a single file, or in a few files, never one file per term;
- in the core, posting lists may be stored uncompressed, as fixed-width binary integers, e.g., with array or numpy;
- the lexicon, the document table, and the statistics may be stored in any format, pickle and numpy files included.

At query time, the lexicon, the document table, and the statistics may be held in memory. Posting lists must be read from disk, with ordinary file reads or mmap, only for the terms of the current query; do not load them all at start-up, and do not cache decoded posting lists across queries.

C3 — Query processing. Implement ranked conjunctive (AND) and disjunctive (OR) query processing, **document at a time** (Chapter 5), keeping the best k results in a min-heap of size k . Expose it as a function

```
search(query: str, k: int, mode: str, scoring: str) -> list[tuple[int, float]]
```

where mode is "and" or "or", scoring is "tfidf" or "bm25", and the result is a list of at most k pairs (pid, score) in decreasing order of score. The instructor may call this function with new queries during the project discussion.

Query semantics:

- the query is processed with the same function as the passages (C1); each distinct resulting term is counted once;
- **OR**: a document is a candidate if it contains at least one query term; terms missing from the lexicon are ignored;
- **AND**: a document is a candidate if it contains every query term; if any term is missing from the lexicon, the result is empty;
- a query with no terms left after processing has an empty result;
- ties in score are broken by increasing docid.

Scoring functions, where q is the set of distinct query terms, $\text{tf}(t, d)$ the frequency of t in d , and $\text{df}(t)$ the number of documents containing t .

- **TF-IDF**:

$$\text{TFIDF}(q, d) = \sum_{t \in q \cap d} (1 + \log \text{tf}(t, d)) \cdot \log \frac{N}{\text{df}(t)}$$

- **BM25**:

$$\text{BM25}(q, d) = \sum_{t \in q \cap d} \log \left(1 + \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} \right) \cdot \frac{(k_1 + 1) \text{tf}(t, d)}{\text{tf}(t, d) + k_1 (1 - b + b |d| / |d|_{\text{avg}})}$$

with k_1 and b as parameters, defaulting to $k_1 = 0.9$ and $b = 0.4$.

C4 – Evaluation. Produce the runs listed in [Section 7.1](#), with $k = 1000$, and evaluate them following [Section 7](#).

C5 – Testing. Regression tests on a toy collection of a few documents written by you, covering at least:

- document processing on hand-picked strings, including punctuation, non-ASCII characters, and malformed lines;
- index construction: lexicon, posting lists, and document table must match values computed by hand;
- merging: the final index built with a buffer so small that it forces several intermediate runs must be identical to the one built with a single run;
- query processing: for both modes and both scoring functions, `search` must return the same results as a brute-force implementation that scores every document of the toy collection directly from the formulas.

Tests may use plain `assert` statements. They must run in a few minutes and must not require the full collection.

5.2 Extensions (optional)

Extensions are **not required**: a correct and complete core is enough to pass. Extensions contribute to higher marks. As a guideline, a team aiming at the maximum grade should complete at least 3 extensions.

Depth and quality of analysis matter more than the number of extensions; extensions do not compensate for a weak core.

General rules for extensions:

- extensions are evaluated with BM25; applying them to TF-IDF is not required;
- for each extension, compare it with the core system as described below, and include the comparison in the report ([Section 8](#));

- where an extension must return the *same results* as the core, the pids and their order must be identical, and scores equal up to 10^{-6} , with ties broken by increasing docid;
- “query length” means the number of distinct terms after processing.

E1 — Index compression. Store docids as gaps (d-gaps) and compress gaps and term frequencies with an integer code implemented by you, e.g., variable-byte encoding (Chapter 4). Keep a flag that writes the index either in a human-readable text format, for debugging on the development subset, or in compressed binary format. Report the size of the posting file and the query latency of the BM25 OR run on dev (small), with and without compression.

E2 — Skipping and dynamic pruning. Add skip pointers to the posting lists and implement a `nextGEQ(d)` operation on your posting-list interface, which moves to the first posting with `docid ≥ d`. Use it to speed up conjunctive queries. For disjunctive queries, implement MaxScore or WAND, storing each term’s maximum BM25 score in the lexicon. Pruning must be *safe*: results must be the same as without pruning. On dev (small), report latency and number of documents scored, for $k \in \{10, 100, 1000\}$ and as a function of query length.

E3 — Term-at-a-time processing. Implement term-at-a-time processing (Chapter 5) of conjunctive and disjunctive queries, with one accumulator per candidate document. It must return the same results as your document-at-a-time implementation. On dev (small), compare the two strategies in latency and memory (number of accumulators), for $k \in \{10, 100, 1000\}$ and as a function of query length, and explain when each is preferable.

E4 — Further integer codes and dictionary compression. Requires E1. Implement at least two further codes among gamma, Golomb, and Rice, and one block-based scheme from Chapter 4, e.g., frame-of-reference. Compress the lexicon with blocking and front coding. Compare the size of the posting file and of the lexicon, and the decoding speed, for every scheme.

E5 — Positional index and phrase queries. Store in each posting the positions of the term in the document, counted in the token stream before stop-word removal. Support phrase queries written between double quotes, e.g., “new york”: a document matches if it contains the phrase terms at consecutive positions, ignoring removed stop words; matching documents are ranked by BM25. Report the index-size overhead of positions, and compare the latency of 100 phrase queries with that of the same queries processed as AND queries. Build the phrase queries from word pairs occurring in dev (small) queries.

E6 — Pseudo-relevance feedback. Implement RM3 or Rocchio (Chapter 10) on top of BM25 OR. Tune the number of feedback documents (at least $\{5, 10, 20\}$), the number of expansion terms (at least $\{10, 20, 50\}$), and the interpolation weight (at least $\{0.3, 0.5, 0.7\}$) on TREC DL 2019 by `nDCG@10`, then evaluate the tuned configuration on TREC DL 2020. Compare with plain BM25 on both.

E7 — Parameter tuning. Study the sensitivity of BM25 to its parameters with a grid search over at least $k_1 \in \{0.6, 0.9, 1.2, 1.5, 2.0\}$ and $b \in \{0.2, 0.4, 0.6, 0.75, 0.9\}$. Tune on TREC DL 2019 by `nDCG@10`, then evaluate the tuned parameters on TREC DL 2020 and dev (small), and discuss whether tuning generalises. Plot `nDCG@10` over the grid.

Proposing your own extension is welcome, but it must be approved by the instructor in advance.

6. Notebook structure

Submit a single notebook, organised in this order:

1. **Set-up:** package installation with pinned versions (`pip install package==x.y.z`), Google Drive mounting, and one configuration cell holding all paths and flags (stemming, stop-word removal, buffer size, BM25 parameters, ...). Print the runtime characteristics: Python version, number of CPU cores, RAM.

2. **Data:** download of the collection, queries, and judgements.
3. **Implementation:** document processing, indexing, query processing, scoring, and extensions. Longer code may be written to modules with `%%writefile` from within the notebook and imported.
4. **Tests** (C5).
5. **Indexing runs**, printing time, peak memory, and index size.
6. **Evaluation and experiments**, producing every table and plot of the report.
7. **Report** (Section 8).

The notebook must run from top to bottom on a fresh free runtime, after editing only the configuration cell. Steps that take long, such as indexing, must save their output to Google Drive and must be skipped when that output already exists. Submit the notebook **with its outputs saved**, so that results can be inspected without running it; keep outputs short, and do not print millions of lines.

Run files use the standard TREC format, one line per retrieved passage:

```
<qid> Q0 <pid> <rank> <score> <run-tag>
```

where ranks start at 1, scores are in decreasing order, and the run tag is a name without spaces identifying the configuration. A query with no results has no lines. Run files stay on your Drive and are not submitted.

7. Experimental protocol

7.1 Required runs

Build two indexes of the full collection: one with stemming and stop-word removal, one with neither. Then produce the following runs, with $k = 1000$:

Run	Scoring	Mode	Index	Query sets
R1	BM25	OR	stemming and stop-word removal	DL 2019, DL 2020, dev (small)
R2	TF-IDF	OR	stemming and stop-word removal	DL 2019, DL 2020, dev (small)
R3	BM25	AND	stemming and stop-word removal	DL 2019, DL 2020, dev (small)
R4	BM25	OR	no stemming, no stop-word removal	DL 2019, DL 2020

Extensions add their own runs (Section 5.2).

7.2 Effectiveness

Compute the following measures with `ir_measures`:

Query sets	Measures	<code>ir_measures</code> names
TREC DL 2019 and 2020	nDCG@10, average precision, Recall@1000	nDCG@10, AP($rel=2$), R($rel=2$)@1000
MS MARCO dev (small)	MRR@10, Recall@1000	RR@10, R@1000

On TREC DL, $nDCG@10$ uses the graded judgements; average precision and recall treat levels 2 and 3 as relevant, as in the official TREC evaluation.

`ir_measures`, like `trec_eval`, averages only over the queries that appear in the run file. Queries with no results, which are frequent in AND mode, must count as 0: compute per-query values with `ir_measures.iter_calc`, and average them yourself over **all** judged queries of the set.

7.3 Efficiency

- **Indexing time:** wall-clock time from reading the first line of the collection to writing the final index, excluding the download. Report separately the time to build the intermediate runs and the time to merge them.
- **Peak memory:** peak resident memory of the indexing process, e.g., with `resource.getrusage(resource.RUSAGE_SELF).ru_maxrss` (in kilobytes on Linux). If you index inside the notebook kernel, restart the runtime first, so that earlier cells do not inflate the peak.
- **Index size:** size on disk, in bytes, of each component: posting file(s), lexicon, document table.
- **Query latency:** time of each call to search, measured with `time.perf_counter()`, including query processing and reading the posting lists, excluding index loading and writing the run file. Before timing, run the whole query set once to warm up; for dev (small), its first 500 queries. Query processing must be single-threaded. Report mean, median, and 95th percentile, in milliseconds, for every run of [Section 7.1](#).

7.4 Comparisons and significance

Required comparisons:

1. TF-IDF versus BM25 (R2 versus R1);
2. conjunctive versus disjunctive processing, in effectiveness and latency (R3 versus R1);
3. the effect of stemming and stop-word removal on effectiveness, index size, and latency (R4 versus R1);
4. one comparison for each extension you implement ([Section 5.2](#)).

For each comparison, run a two-sided paired t-test (`scipy.stats.ttest_rel`) on the per-query values of $nDCG@10$ on each TREC DL set, and of $MRR@10$ on dev (small) where available, and report the p -values. Use a significance level of 0.05, and remember that 43 or 54 queries give tests of limited power.

7.5 Sanity check

Run R1 should obtain roughly $nDCG@10 \approx 0.50$ on TREC DL 2019, ≈ 0.48 on TREC DL 2020, and $MRR@10 \approx 0.18$ on dev (small). Deviations of a few hundredths are expected, because of differences in tokenisation and stop words; larger ones usually indicate a bug (see [Section 10](#)).

8. Deliverables

Submit the **link to your Colab notebook**, shared so that anyone with the link can view it. Nothing else is required: code, tests, results, and report all live in the notebook.

Report. The last part of the notebook, written in text cells, in English. There is no strict length limit; as a guideline, aim at the equivalent of about 10 pages. Sections:

1. what your program can do;
2. how it works internally: architecture, major functions and modules, data structures and file layout, and the reasons for your design decisions;
3. experimental set-up: runtime characteristics, data, protocol;

4. results: how long it takes to index and to process queries, how large the index is, and how effective it is, with the comparisons of [Section 7.4](#);
5. extensions: which ones you implemented, and their comparisons;
6. limitations, and what you would do differently;
7. contribution of each team member, and declared use of AI assistants.

9. Final evaluation

The final evaluation consists of the project discussion and the oral examination, according to the following rules:

1. every student discusses the project individually, independently of the other team members;
2. a student may take the oral examination in the same exam session as the project discussion or in a later one;
3. a project can be discussed in an exam session only if at least one team member also takes the oral examination in that session: project discussions with no oral examination are not held;
4. a student who fails the final evaluation must take the oral examination again in a later exam session, but does not repeat the project discussion;
5. the notebook link must be communicated by e-mail **no later than three working days** before the official date of the first exam session in which the project is discussed; late communications are not taken into account. For example, for an exam session on a Monday, the deadline is the Wednesday before;
6. the notebook is frozen at that deadline: changes made afterwards are not taken into account, also when other team members discuss the project in later exam sessions; the Colab revision history is used to check this.

The e-mail. One e-mail per team, sent to the instructor with all team members in copy, with subject [MIRCV 2026] Project – <surnames of the members>, containing:

- name, surname, student number, and e-mail address of each member;
- the notebook link of **both projects**;
- the list of extensions implemented for each project.

Every student who discusses the project or takes the oral examination must also register for that exam session through the university's exam registration system.

The project discussion includes questions on your code and questions on the theory behind the components you implemented, following the learning objectives in [Section 2](#).

10. FAQ

Can I use a subset of the collection? Only for development. All results in the report must be on the full collection.

Do I need a GPU or a paid Colab plan? No. A free CPU runtime is enough, and everything you report must be reproducible on one ([Section 3](#)).

My indexing takes hours. Profile first on the development subset. Common causes are per-posting Python objects, dictionaries holding millions of small values, repeated string concatenation, and unbuffered writes. Cache the stem of each distinct token instead of stemming every occurrence; keep the document table in array or numpy arrays rather than in dictionaries; write posting lists as binary arrays. Compile the hottest loops if needed ([Section 3](#)).

My Colab session disconnected during indexing. Make indexing restartable: write each intermediate run to Google Drive as soon as it is complete, and skip the runs that already exist.

My Google Drive is full. Keep the collection only on the runtime's local disk, re-downloading it when needed, and delete intermediate runs once the final index is written.

Queries are slow. Check that the index is read from local disk, not from Google Drive ([Section 4.3](#)), and that you only read the posting lists of the query terms.

Must docids be equal to pids? No. They may coincide, but the document table must store the mapping, and run files must contain pids ([Section 4.1](#)).

Many AND queries return no results. Is that a bug? Not necessarily: long queries often have no document containing all their terms. These queries count as 0 in the averages ([Section 7.2](#)).

Can I use pickle? Yes, for the lexicon, the document table, and the statistics; not for the posting lists ([Section 5.1](#), C2).

Can I use multiple processes? For indexing, yes: state the number of processes in the report. Query processing must be single-threaded ([Section 7.3](#)).

Must extensions also work with TF-IDF? No, BM25 is enough ([Section 5.2](#)).

My R1 results are far from the sanity check. Check, in this order: that queries go through the same processing as passages; the IDF formula ([Section 5.1](#), C3); that document lengths are computed after processing; that queries without results count as 0; that run files contain pids, not docids; that TREC DL average precision and recall use $rel=2$.

Can I reuse code from the lab sessions? Yes, if you cite it in the report.

Can I implement some parts in a separate notebook? No: a single notebook contains everything ([Section 6](#)).

Can we change the team composition? Not after the notebook link has been communicated. If a member withdraws and the team falls below three students, contact the instructor by e-mail.